Chapter 1

Reinforcement Learning in Finance

1.1 Introduction

Every model built so far in this book, however sophisticated, has answered a single question once. Chapter 6's classifiers predict a label from a feature vector; Chapter 9's credit models predict a default probability; Chapter 13's Almgren-Chriss trajectory prescribes an entire trading schedule, but it does so by solving one optimization problem in advance and then executing the resulting plan. None of these approaches change their answer in response to how the world actually responds to the decisions already made.

A portfolio manager, a market maker, and a trading desk do not operate this way: they observe an outcome, update their beliefs, and choose their next action accordingly, over and over, for as long as the position stays open. Reinforcement learning is the branch of machine learning built specifically for this setting, sequential decision-making under uncertainty, where an agent's own actions influence the future states it will face, and where the right action today depends on the consequences it sets up for tomorrow, not only on today's immediate reward.

This chapter formalizes that setting, the Markov decision process, and develops the two families of algorithms built to solve it: value-based methods, culminating in Q-learning and its neural-network extension, and policy-based methods, culminating in the policy-gradient algorithms that power most modern applications. Rather than introducing these ideas in the abstract and only later connecting them to finance, this chapter builds directly on machinery this book has already developed: Chapter 5's binomial-tree early-exercise decision, Chapter 12's portfolio rebalancing problem, and Chapter 13's optimal-execution and market-making models are all, on inspection, special cases of the general sequential-decision problem this chapter formalizes, ones simple enough that a closed-form or backward-induction solution exists. Reinforcement learning is what a practitioner reaches for once the problem is still sequential but no longer simple enough for a closed form, a genuinely common situation once market impact, transaction costs, and a client's actual risk preferences no longer fit the clean functional forms those earlier chapters assumed.

The chapter's central worked example makes this connection concrete rather than asserting it. It reframes Chapter 5's American put early-exercise decision as a three-state Markov decision process, solves it with model-free Q-learning instead of backward induction, and shows the learned answer converges to the same value the tree already established. That is a small, fully verifiable proof that reinforcement learning recovers a known-correct answer, before this chapter asks the reader to trust it on problems that have no closed form at all.

The chapter proceeds in three stages. Sections 1.2.1 through 1.2.5 build the general Markov decision process framework, the Bellman equation, and the vocabulary, model-based versus model-free, on-policy versus off-policy, needed to place every algorithm that follows. Sections 1.3.1 through 1.3.3 develop the algorithms themselves, starting with the fully verifiable tabular Q-learning worked example and building up to deep Q-networks and policy-gradient methods for problems too large for a table. The remaining sections turn to application and honest assessment: optimal execution, market making, and portfolio management (Sections 1.4.1 through 1.4.4), a real industry case study (Section 1.5.1), and the evaluation methodology and risks (Sections 1.5.2 and 1.6) that separate a reinforcement-learning system that is merely impressive in a backtest from one that is actually safe to deploy with real capital.

Wulin.Suo@Queensu.ca

1.2 The Reinforcement Learning Framework

This part sets up the sequential decision problem and the value functions that make it tractable.

1.2.1 The Reinforcement Learning Framework: States, Actions, Rewards, and Policies

Consider a market maker deciding, moment by moment, how to skew its bid and ask quotes as its inventory evolves, or a trader deciding how many shares of a large order to release this minute versus holding back for a better price later: in both cases, today's decision changes tomorrow's starting position, so the two decisions cannot be solved separately, one at a time, the way most of the models in Chapters 12 and 13 do. A reinforcement learning problem is formalized as a Markov decision process, a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, precisely to capture this kind of interlocking, path-dependent decision problem.

At each discrete time step t, an **agent** observes a **state** $s_t \in \mathcal{S}$, a summary of everything about the environment relevant to the decision at hand (a market maker's current inventory and the prevailing bid-ask spread; a trading agent's remaining order size and elapsed time; a portfolio manager's current holdings and estimated risk factors). The agent selects an **action** $a_t \in \mathcal{A}$ (post a quote at a given skew; sell a given number of shares this period; rebalance toward a given target weight) according to a **policy** $\pi(a \mid s)$, which gives the probability of taking action a in state s and so maps each state to a probability distribution over the available actions, with $\sum_a \pi(a \mid s) = 1$. A policy that places all of its probability on one action per state is deterministic, the special case where the agent always does the same thing from the same state.

The environment then transitions to a new state s_{t+1} according to a transition function $P(s_{t+1} \mid s_t, a_t)$ and returns a scalar **reward** r_{t+1} (realized profit and loss net of costs; a small quadratic risk penalty; an exercise payoff), governed by a reward function $R(s_t, a_t, s_{t+1})$. Figure 1.1 depicts this loop; it repeats until the episode ends, either at a fixed horizon (the market closes, the order is fully executed) or when the agent reaches an absorbing state (a position is liquidated, an option is exercised).

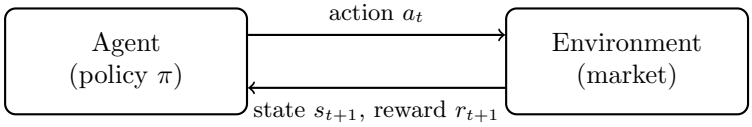


Figure 1.1: The reinforcement learning loop

The word "Markov" in Markov decision process is a precise mathematical assumption, not a figure of speech: it requires that $P(s_{t+1} \mid s_t, a_t)$ and $R(s_t, a_t, s_{t+1})$ depend only on the current state and action, not on the full history that led there. This is the assumption behind every stochastic process this book has already used to model prices, from the binomial tree's independent up-and-down moves in Chapter 5 to the random-walk and GARCH innovations of Chapter 7, so it is not a new leap of faith for a reader of this book, only a new name for a familiar structural assumption, now applied to an agent's decisions rather than only to prices themselves. In practice, whether a proposed state representation is genuinely Markovian is itself a design choice: a market maker's state that includes only the current mid-price, and not recent order flow, discards information a real market maker would use, which is why Chapter 14's order-flow-imbalance regression and Chapter 13's Kyle's-lambda framework are natural sources of state features for a genuinely useful trading agent, not merely of standalone predictive signals.

The agent's goal is to choose a policy that maximizes the expected **return**, the discounted sum of future rewards,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

where the **discount factor** $\gamma \in [0, 1]$ plays a role directly analogous to the discount factor $1/(1+r)$ used everywhere in Chapter 5's valuation formulas: a reward received later is worth less today, both because of the time value of money and, in the reinforcement learning setting, because a reward far in the future is a noisier, harder-to-credit consequence of the current action. A myopic agent with $\gamma = 0$ maximizes only the immediate reward, the one-shot optimization every earlier chapter's models perform; the closer γ is to

Wulin.Suo@Queensu.ca

1, the more the agent's current choice is shaped by consequences many steps away, which is precisely the additional structure a sequential problem like optimal execution or ongoing portfolio management requires and a one-shot model cannot represent.

1.2.2 From One-Shot Optimization to Sequential Decisions

It is worth being explicit about why the models developed in Chapters 12 and 13 did not need this machinery, since the answer clarifies exactly what reinforcement learning adds. The Almgren-Chriss optimal-execution model (Chapter 13's Section 13.3.5) assumes a known, fixed functional form for market impact, quadratic in the trade rate, and a known, fixed risk-aversion parameter ϕ; given those assumptions, the entire optimal trajectory can be solved in closed form in one step, with no need to learn anything from experience, because the model already specifies exactly how the environment will respond to every possible action. Chapter 13's Avellaneda-Stoikov inventory model (Section 13.4) is more genuinely dynamic, the market maker's quotes are recomputed at every instant as inventory and time-to-close evolve, but the update rule is itself a closed-form formula derived from an assumed order-arrival process, not a policy learned from trial and error. Chapter 12's Section 12.4 practical-allocation discussion goes furthest toward acknowledging the sequential nature of real portfolio management, noting that rebalancing decisions recur over time and that transaction costs make each rebalancing decision depend on the portfolio's current position, not only on today's estimated optimal weights, but it stops short of solving that sequential problem directly, instead treating each rebalancing decision as a separate, myopic optimization. Each of these models is exactly right within its assumptions, and each remains the correct tool to reach for whenever those assumptions hold: a known, well-estimated impact function and a well-specified risk-aversion parameter make a closed-form solution both more accurate and vastly cheaper to compute than a learned one.

Reinforcement learning becomes the more appropriate tool precisely when that closed form is unavailable, because the true market-impact function is not well approximated by any simple parametric form, because transaction costs interact with the portfolio's history in ways too complex to solve analytically, or because the reward the agent actually cares about, a client's realized utility net of taxes, fees, and behavioral tolerance for drawdowns, does not reduce cleanly to a quadratic cost function at all. In every one of these harder cases, the underlying problem is still a sequential decision problem exactly as formalized in Section 1.2.1; what changes is only that no one can write down P and R in closed form, which rules out the direct optimization Chapters 12 and 13 rely on and motivates learning a good policy from simulated or historical experience instead.

1.2.3 The Simplest Case: Multi-Armed Bandits

A trading desk sending orders to market has a choice most descriptions of execution quietly skip past: which venue to send each order to. A large-cap U.S. equity trades on a dozen or more venues, and the same order routed to two of them will come back with different realized prices, because each venue has its own mix of resting liquidity, its own fee schedule, and its own population of counterparties. A desk that knew each venue's average price improvement would simply route everything to the best one. No desk knows this. Worse, the only way to find out what a venue delivers is to send it orders, which means the desk is paying, in worse fills, for every order it spends learning rather than earning.

This is the tension that separates reinforcement learning from everything in Chapters 6 and 7. A supervised model is handed a fixed dataset and never influences what it gets to see next; here, the agent's own choices determine which data it ever observes, so an action that looks bad early may simply never be tried again, and its true value never discovered. The problem in its purest form is the **multi-armed bandit**, named for a row of slot machines with unknown payout rates: at each step the agent picks one of K actions, called **arms**, and receives a noisy reward drawn from that arm's own fixed but unknown distribution.

What makes the bandit the right place to start is everything it leaves out. There is one state, which never changes; the reward depends only on the action just taken, never on history; and no action moves the agent anywhere, so there is no next state to look ahead to. The entire apparatus of Section 1.2.4's Bellman equation, discounting, bootstrapping, and credit assignment across a trajectory, is unnecessary here, because $Q(a)$ is nothing more elaborate than the mean reward of arm a, estimable by averaging. The only genuine

Wulin.Suo@Queensu.ca

difficulty left is the one this chapter cannot avoid anywhere: the **exploration-exploitation trade-off**, choosing between the arm that currently looks best and one whose value is still uncertain.

A Worked Example: Routing Orders Across Three Venues

A desk routes each arriving order to one of three venues. Measured in basis points of price improvement against the arrival mid-price, venue A delivers a mean of 1.2, venue B a mean of 2.0, and venue C a mean of 0.5, each with a standard deviation of 2.0 basis points. Those means are stated here so the example can be checked; the desk does not know them, and must infer them from the fills it receives. Since the standard deviation of a single fill is 2.5 times the 0.8 basis-point gap between the best venue and the second best, one order tells the desk almost nothing.

The desk keeps an estimate $Q(a)$ of each venue's mean and a count $N(a)$ of orders sent there, updating after each fill by the running-average rule

$$Q(a) \leftarrow Q(a) + \frac{1}{N(a)}\big[r - Q(a)\big],$$

which is exactly the sample mean of the rewards seen from a, written so it can be updated one fill at a time without storing them. After sending one order to each venue to get started, it follows an ε-greedy policy with $\varepsilon = 0.2$: with probability 0.2 route to a uniformly random venue, otherwise route to whichever venue currently has the highest Q. Table 1.1 traces the first twelve orders.

Order	Rule	Venue	Fill	$Q(A)$	$Q(B)$	$Q(C)$
1	warm-up	A	1.45	1.450	—	—
2	warm-up	B	1.74	1.450	1.740	—
3	warm-up	C	1.78	1.450	1.740	1.780
4	explore	A	1.92	1.685	1.740	1.780
5	exploit	C	2.39	1.685	1.740	2.085
6	exploit	C	−2.03	1.685	1.740	0.713
7	exploit	B	2.08	1.685	1.910	0.713
8	exploit	B	1.56	1.685	1.793	0.713
9	exploit	B	0.54	1.685	1.480	0.713
10	exploit	A	0.57	1.313	1.480	0.713
11	exploit	B	4.09	1.313	2.002	0.713
12	explore	C	3.23	1.313	2.002	1.342

Table 1.1: The first twelve orders of the venue-routing bandit. Fills and estimates are in basis points of price improvement; a dash marks a venue not yet tried. Every estimate follows from the running-average rule applied to the fills above it, so each row can be checked by hand.

The opening rounds show precisely the failure mode that makes exploration necessary. The single warm-up order to venue C returned 1.78 basis points, a good fill drawn by chance from the *worst* of the three venues, and that one lucky draw was enough to make C the greedy choice. Orders 5 and 6 duly went there. What rescued the desk was not a clever exploration rule but simple repetition: order 6 came back at -2.03, dragging $Q(C)$ from 2.085 down to 0.713 and demoting C below both rivals. Exploiting a wrong estimate long enough tends to correct it, because every exploit is also a sample. By order 12 the desk has $Q(B)$ highest and is routing to the genuinely best venue, though $Q(A) = 1.313$ and $Q(C) = 1.342$ remain badly estimated off three and four orders respectively.

Run the same policy for 5,000 orders and the estimates sharpen to $Q(A) = 1.25$, $Q(B) = 1.99$, and $Q(C) = 0.40$, against true means of 1.2, 2.0, and 0.5. The desk routes 86.2% of its orders to B and realizes an average of 1.826 basis points per order.

That average is the point worth dwelling on, because it is visibly short of the 2.0 basis points a desk that simply knew the answer would earn. The shortfall, 868 basis points in total across the 5,000 orders, is the **regret**: what learning cost, measured against an oracle that never had to learn. Most of it is not the early

Wulin.Suo@Queensu.ca

mistakes, which cost little and are over quickly. It is the ε-greedy rule itself, which keeps routing one order in five at random forever, long after the answer is no longer in doubt. That permanent tax is exactly

$$\varepsilon \times \big(2.0 - \tfrac{1}{3}(1.2 + 2.0 + 0.5)\big) = 0.2 \times 0.7667 = 0.1533 \text{ basis points per order,}$$

which is why the asymptotic average is $0.8 \times 2.0 + 0.2 \times 1.2333 = 1.8467$ rather than 2.0, and why the realized 1.826 sits just below it. The remedy is to explore hard early and taper off: decaying ε over the same 5,000 orders lifts the share routed to B from 86.2% to 95.2% and the average fill from 1.826 to 1.943 basis points, without giving up the early exploration that found B in the first place. Every worked example later in this chapter uses a decayed exploration schedule for this reason.

Two threads run forward from here. Add a next state, so that today's action changes tomorrow's situation, and averaging is no longer enough: the agent must reason about consequences that arrive later, which is what Section 1.2.4's Bellman equation supplies and what Section 1.3.1's Q-learning example puts to work. Keep the single state but shrink the gap between arms relative to the noise, and even this simplest of problems becomes genuinely hard: Section 1.4.2 returns to a bandit whose best and second-best arms differ by so little against so much noise that 200,000 rounds still leave the agent on the wrong one.

1.2.4 Value Functions and the Bellman Equation

Suppose two agents follow the exact same policy but happen to start an episode from different states, one with a small, comfortable inventory and one already deep in an overextended position: the policy alone cannot say how much trouble each agent is actually in, only what each plans to do next from wherever it stands. Two functions summarize how good a given state, or state-action pair, is under a policy π, answering this question: the **state-value function** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$, the expected return starting from state s and following π thereafter, and the **action-value function** $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$, the expected return from taking action a in state s and following π afterward.

Because the return decomposes into an immediate reward plus a discounted continuation, both functions satisfy a recursive relationship, the **Bellman equation**, named for Richard Bellman's original work on dynamic programming:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\Big[R(s,a,s') + \gamma V^\pi(s')\Big].$$

The **Bellman optimality equation** specializes this to the best possible policy, replacing the expectation over actions with a maximization:

$$Q^*(s,a) = \sum_{s'} P(s' \mid s, a)\Big[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\Big].$$

A reader who worked through Chapter 5's two-step binomial tree (Section 5.4.1) has already solved a Bellman optimality equation by hand without that name attached to it. At each node, the tree's backward-induction step compares the value of exercising immediately against the discounted expected value of continuing, and takes whichever is larger. That is the $\max_{a'}$ in the equation above, applied to the two actions "exercise" and "hold." This is not a loose analogy; Section 1.3.1 makes the correspondence exact by reformulating that identical tree as a Markov decision process and re-deriving the same option value from it. Dynamic programming methods, value iteration and policy iteration, solve the Bellman equation directly whenever P and R are fully known, exactly as Chapter 5's tree does. Reinforcement learning's model-free methods, developed next, instead estimate Q^* purely from sampled transitions and rewards, without ever needing P and R written down explicitly. That property is what makes them applicable to problems, like most real trading and portfolio problems, where no one can specify P and R in closed form to begin with.

1.2.5 A Taxonomy of Reinforcement Learning Methods

Two further distinctions organize the wide range of reinforcement learning algorithms in use today, and fixing both precisely now makes it easier to place every method the rest of this chapter introduces. Figure 1.2

Wulin.Suo@Queensu.ca

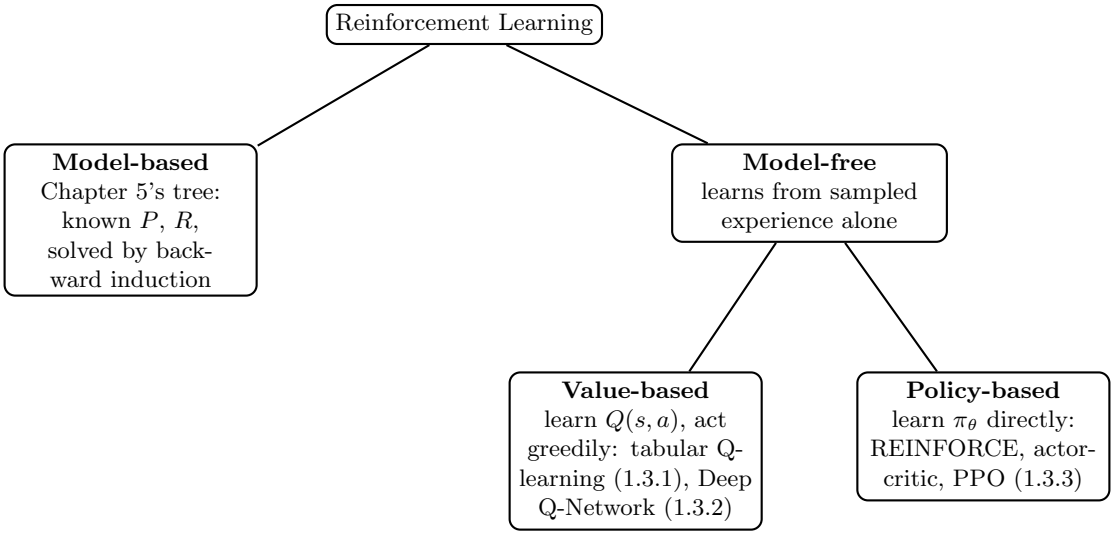


Figure 1.2: A map of the reinforcement learning methods this chapter develops

previews where each of them fits before the text below explains why; π_θ there denotes a policy carrying tunable parameters θ, the form Section 1.3.3 develops.

Model-based methods have access to, or explicitly learn, the transition function P and reward function R themselves, and use them to plan. Chapter 5's binomial tree is a model-based method: q, u, d, and every terminal payoff are known in advance, so backward induction can compute the exact optimal value with no simulated experience required at all. **Model-free methods** never estimate P or R directly; they learn a value function or a policy purely from sampled transitions and rewards, treating the environment as a black box that can be queried (by simulating an episode) but not inspected. Every method developed in the remainder of this chapter, tabular Q-learning, deep Q-networks, and the policy-gradient family, is model-free, which is precisely why each remains usable in the realistic settings Section 1.2.2 described, where no closed-form P and R exist for a real limit order book or a real client's multi-period utility.

A third, hybrid category, sometimes called **learned-model-based methods**, trains an approximate model of P and R directly from data (rather than assuming it, as Chapter 5's tree does) and then plans against that learned model, gaining some of model-based planning's sample efficiency without requiring the transition and reward functions to be known in closed form in advance. This hybrid approach underlies some of reinforcement learning's most prominent successes outside finance, including the game-playing systems that combine a learned model with tree search, but it introduces its own risk, the sim-to-real gap Section 1.5.2 discusses, since a plan is only as good as the learned model it is built against.

Off-policy methods learn about a policy different from the one actually generating their training data. Q-learning's update rule is a clear example: its bootstrapped target uses $\max_{a'} Q(s', a')$, the value of the best available action, regardless of which action the ε-greedy behavior policy, one that takes a random action with probability ε and the currently best-valued action otherwise, actually selected next. This lets a single stream of exploratory, partly random experience be reused to learn about the greedy-optimal policy even while the agent is still busy exploring, exactly why Section 1.3.1's worked example converges to the correct answer despite spending a meaningful fraction of its 100,000 episodes taking deliberately random actions. Off-policy methods have a clear data-efficiency advantage, one batch of experience can be reused across many rounds of learning, exactly why DQN's replay buffer (Section 1.3.2) works at all.

On-policy methods instead evaluate and improve the policy currently being followed. The classic on-policy analogue of Q-learning, SARSA (State-Action-Reward-State-Action), differs only by replacing $\max_{a'} Q(s', a')$ with $Q(s', a')$ for the action a' the policy actually selects next, so its learned values reflect the actual, still-exploring policy's behavior rather than the ultimate greedy target. Policy-gradient methods (Section 1.3.3) are typically on-policy as well, since their gradient estimator is only valid for returns actually experienced under the current policy π_θ, not under some other, older policy. On-policy methods are generally

Wulin.Suo@Queensu.ca

simpler to reason about and more stable to train than off-policy methods, since they never have to correct for a mismatch between the policy that generated the data and the policy currently being evaluated.

1.3 Methods

From a lookup table to a neural policy, this part builds the algorithms in order of the problems that forced each one.

1.3.1 A Worked Example: Q-Learning Recovers Chapter 5's Early-Exercise Decision

Setting Up the Markov Decision Process

Chapter 5's American put example used a two-step binomial tree with $S_0 = K = \$50$, up and down factors $u = 1.2$ and $d = 0.9$, and a gross risk-free return $R = 1.04$ per period, giving a risk-neutral up-probability

$$q = \frac{R - d}{u - d} = \frac{0.14}{0.30} \approx 0.4667$$

and a per-step discount factor $\gamma = 1/R \approx 0.9615$.

That tree has exactly three decision points: today, at $S_0 = \$50$; after one up move, at $S_u = \$60$; and after one down move, at $S_d = \$45$. At each of these three states the holder chooses between two actions: **exercise**, receive the intrinsic value $\max(K - S, 0)$ immediately and end the episode, or **hold**, receive nothing now and let the tree evolve one more step.

The two terminal nodes reachable from the up state, $S_{uu} = \$72$ and $S_{ud} = \$54$, both pay $\max(50-S, 0) = \$0$; of the two reachable from the down state, S_{ud} again pays \$0, while $S_{dd} = \$40.50$ pays $\max(50-40.50, 0) = \$9.50$.

Putting these pieces together, this is precisely a Markov decision process with states $\mathcal{S} = \{S_0, S_u, S_d\}$ and actions $\mathcal{A} = \{\text{exercise}, \text{hold}\}$. Transition probabilities are given by q and $1-q$, rewards by the intrinsic-value and terminal-payoff structure above, and $\gamma = 1/R$ discounts every step exactly as Chapter 5's tree already did.

Figure 1.3 redraws Chapter 5's tree in this chapter's language, labeling each decision node with its two available actions rather than with a single option value.

Chapter 5 solved this exact problem by backward induction, working from the known terminal payoffs back to today. Restated in this chapter's notation, that solution is

$$\begin{aligned}
Q^*(S_u, \text{exercise}) &= \$0.00, & Q^*(S_u, \text{hold}) &= \gamma\big[q(0) + (1-q)(0)\big] = \$0.00,\\
Q^*(S_d, \text{exercise}) &= \$5.00, & Q^*(S_d, \text{hold}) &= \gamma\big[q(0) + (1-q)(9.50)\big] \approx \$4.8718,\\
Q^*(S_0, \text{exercise}) &= \$0.00, & Q^*(S_0, \text{hold}) &= \gamma\big[q\,V^*(S_u) + (1-q)\,V^*(S_d)\big] \approx \$2.5641,
\end{aligned}$$

where $V^*(S_u) = \max(0.00, 0.00) = \$0.00$ and $V^*(S_d) = \max(5.00, 4.8718) = \$5.00$, since exercising is optimal at the down node. The optimal policy is therefore to hold at S_0 and S_u and exercise at S_d, giving an American put value of $V^*(S_0) \approx \$2.5641$, the value Chapter 5 reports.

Tabular Q-Learning

Backward induction requires knowing q, γ, and every terminal payoff in advance, the closed-form knowledge Section 1.2.2 argued is unavailable in most real trading and portfolio problems. **Q-learning** instead estimates Q^* from simulated experience alone, never using q directly in its update rule.

Starting from an arbitrary initial table $Q(s, a)$ (here, all zeros), the agent repeatedly simulates an episode: at each state it chooses an action using an ε-greedy policy (with probability ε, act randomly to keep exploring; otherwise, act greedily with respect to the current Q-table), observes the resulting reward r and next state s', and updates

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\Big],$$

Wulin.Suo@Queensu.ca

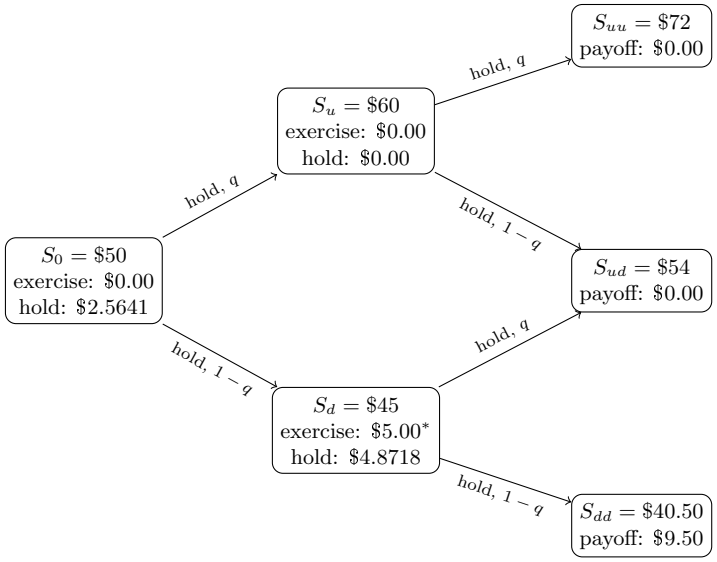


Figure 1.3: The early-exercise decision of Chapter 5's American put, redrawn as a three-state Markov decision process, with the optimal action at each node marked by an asterisk

where α is a learning rate. This update rule, known as **temporal-difference learning**, nudges the current estimate $Q(s, a)$ toward a bootstrapped target, the observed reward plus the discounted value of the best action available in the next state, without ever requiring a model of the transition probabilities.

The agent learns q's effect on the optimal decision purely by observing many simulated up and down moves and seeing which actions paid off, in the same statistical spirit as every other machine learning method in this book learning a relationship from data rather than being told it directly.

A Worked Example: The First Two Learning Steps

Starting from a Q-table initialized entirely to zero, with a learning rate of $\alpha = 0.1$, suppose the very first simulated episode reaches S_d, chooses **hold** (all actions still look identically worthless at this point, so the choice is effectively random), and the simulated down-move occurs, landing on the terminal payoff of \$9.50 at S_{dd}. Since this transition ends the episode, the bootstrapped target's continuation term is zero, and the update is

$$Q(S_d, \text{hold}) \leftarrow 0 + 0.1\big[9.50 + \gamma(0) - 0\big] = 0.1(9.50) = \$0.9500,$$

the first nonzero entry the table ever acquires. Now suppose a later episode reaches S_0, chooses **hold**, and the simulated move is down, transitioning to S_d with an immediate reward of \$0 (holding earns nothing until a terminal payoff is reached). The bootstrapped target now uses the current, already partially learned table, $\max_{a'} Q(S_d, a') = \max(0, 0.9500) = 0.9500$, giving

$$Q(S_0, \text{hold}) \leftarrow 0 + 0.1\big[0 + \gamma(0.9500) - 0\big] \approx 0.1(0.9615)(0.9500) \approx \$0.0913.$$

Two updates in, the table already shows the qualitative shape of the true solution, $Q(S_d, \text{hold})$ noticeably positive and $Q(S_0, \text{hold})$ smaller but nonzero, even though both values are still far from their closed-form targets of \$4.8718 and \$2.5641.

This is the mechanism the 100,000-episode simulation below repeats: each terminal payoff nudges the value of the state that led to it, and that nudge is itself propagated one more step backward the next time an earlier state is visited, gradually diffusing the terminal reward's information all the way back to S_0 without ever consulting q directly.

The companion notebook simulates this process for 100,000 episodes, using a learning rate that decays with each state-action pair's visit count and an exploration rate ε that decays from 0.30 to 0.02 over training, standard choices that guarantee enough early exploration to discover the exercise decision at S_d while still

Wulin.Suo@Queensu.ca

converging to a stable, greedy policy by the end of training. Table 1.2 compares the learned Q-table to the closed-form values derived above.

State-action	Closed-form Q^*	Learned Q (Q-learning)	Relative gap
$(S_0, \text{exercise})$	\$0.0000	\$0.0000	—
(S_0, hold)	\$2.5641	\$2.5328	1.22%
$(S_u, \text{exercise})$	\$0.0000	\$0.0000	—
(S_u, hold)	\$0.0000	\$0.0000	—
$(S_d, \text{exercise})$	\$5.0000	\$5.0000	0.00%
(S_d, hold)	\$4.8718	\$4.9234	1.06%

Table 1.2: Closed-form Bellman-optimal Q-values (from backward induction) versus tabular Q-learning's learned estimates after 100,000 simulated episodes

Comparing the Learned and Closed-Form Solutions

The learned policy matches the closed-form optimal policy exactly: hold at S_0 (\$2.5328 > \$0.0000), exercise at S_d (\$5.0000 > \$4.9234), and, at S_u, both actions are learned as worth \$0.0000, an exact tie that matches Chapter 5's own observation that "both exercise and continuation are worth \$0" at that node, so which action the tie-break happens to favor is economically inconsequential.

Every "exercise" value converges exactly, with no residual error, since exercising ends the episode immediately and its reward is observed without any noise; every "hold" value converges to within roughly 1% of the true value but not exactly, since a "hold" estimate is a bootstrapped average built from a finite number of noisy sampled continuations rather than an exact expectation computed from a known q.

This gap is not a bug to be engineered away; it is the fundamental price of model-free learning, and it is the single most important lesson this worked example offers: when the transition probabilities and payoffs are genuinely known in closed form, as they are in Chapter 5's tree, backward induction is strictly better, exact, instantaneous, and requiring no simulated data at all, and reinforcement learning should not be used. Reinforcement learning earns its keep only in the more realistic setting, developed throughout the rest of this chapter, where no such closed form exists and simulated or historical experience is the only source of information available.

1.3.2 From Tables to Function Approximation: Deep Q-Networks

Tabular Q-learning stores one number for every state-action pair, which works only because Section 1.3.1's example has just three states and two actions. A realistic trading state, some combination of inventory, elapsed time, recent order-flow imbalance (Chapter 14), and a handful of price-based features, takes values from a continuous or extremely large discrete space, making an explicit table impossible to store or to visit often enough to fill in.

The standard fix, introduced by Mnih et al. (2015) and known as the **Deep Q-Network** (DQN), replaces the table with a neural network $Q(s, a; \theta)$, the feedforward architecture Chapter 6's Section 6.6.4 introduced, trained to approximate the same Bellman-optimal Q^* function. The network takes a state (or a fixed-size feature vector describing one) as input and outputs one value per possible action; training minimizes the squared temporal-difference error,

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\big)^2\Big],$$

using the same gradient-descent machinery Chapter 6's Section 6.3.2 already developed for supervised learning, with θ^- a periodically-updated copy of the network's own parameters (a "target network") used to keep the bootstrapped target from chasing a constantly-moving estimate, one of two stabilization tricks, alongside storing and randomly resampling past transitions from a large "replay buffer" rather than training only on the most recent experience, that made deep Q-learning reliable enough to train at all.

Wulin.Suo@Queensu.ca

The underlying update rule is otherwise identical to tabular Q-learning: the network simply generalizes across similar states instead of memorizing each one independently, letting the agent act sensibly in a state it has never visited before, provided that state resembles ones it has seen, the same generalization argument that justifies every other neural network in this book.

Deep Q-networks are also considerably more finicky to train than tabular Q-learning, in a way worth naming explicitly given this book's repeated warnings, in Chapter 12's badly-scaled optimizer example among others, that a plausible-looking numerical method can silently fail to converge without an obvious error message.

A DQN's training loss can decrease steadily while the actual learned policy remains poor, since the loss measures how well the network fits its own, still-inaccurate bootstrapped targets rather than how good the resulting policy actually is. The only reliable check is to evaluate the policy's real performance directly. That is the discipline Section 1.3.1's worked example applied, comparing learned Q-values against an independently known closed-form answer, and the discipline Section 1.5.2 generalizes into a full evaluation methodology for problems where no closed-form answer exists to check against.

1.3.3 Policy Gradient Methods and Continuous Actions

Q-learning and its deep variant both learn a value function and derive a policy from it implicitly, by acting greedily. This works cleanly when the action set is small and discrete, exercise or hold, buy or sell, but breaks down when the natural action is continuous, exactly how many shares to trade this period, or exactly what target weight to assign an asset, since $\max_{a'} Q(s', a')$ has no simple closed form once a' ranges over a continuum.

Policy gradient methods sidestep this by parameterizing the policy directly, $\pi_\theta(a \mid s)$, typically as a neural network outputting the parameters of a continuous distribution (a mean and standard deviation, for instance), and adjusting θ to directly increase the expected return, using the policy gradient theorem,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t\Big],$$

which says, informally, that actions followed by a higher-than-expected return should be made more likely, and actions followed by a lower-than-expected return should be made less likely, with the size of the nudge proportional to how surprising the return was.

Actor-critic methods reduce the noise in this estimate by learning a value function (the "critic") alongside the policy (the "actor") and using it to judge whether a given return was better or worse than expected from that state, rather than judging it against the raw, highly variable return alone.

Proximal Policy Optimization (PPO), introduced by Schulman et al. (2017), is the most widely used modern policy-gradient algorithm. It constrains each policy update to stay close to the previous policy, preventing the large, destabilizing updates that plagued earlier policy-gradient methods, and its combination of reasonable sample efficiency and training stability has made it the default starting point for continuous-action problems across robotics, game-playing, and, increasingly, the trading and portfolio applications developed next.

Table 1.3 summarizes the four families of methods this chapter has introduced, alongside the setting each is best suited to and the concrete finance application, developed in the sections that follow, each is most naturally applied to.

1.3.4 Learning from Expert Demonstrations: Imitation and Inverse Reinforcement Learning

Every method developed so far learns purely from trial and error, an agent that starts knowing nothing about which actions are good must discover this for itself through the ε-greedy exploration of Section 1.3.1 or its neural-network analogues.

When a record of a skilled human decision-maker's past choices already exists, an experienced trader's execution decisions, say, a faster alternative is available: **behavioral cloning** treats the expert's historical state-action pairs as an ordinary supervised training set and fits a policy $\pi_\theta(a \mid s)$ to predict the expert's

Wulin.Suo@Queensu.ca

Method	Learns	Best suited to	Natural finance application
Tabular Q-learning	Action-value function $Q(s, a)$	Small, discrete state and action spaces	Section 1.3.1's early-exercise decision; small discretized inventory or execution problems
Deep Q-Network (DQN)	Neural-network approximation of $Q(s, a; \theta)$	Large or continuous state spaces, discrete actions	Execution and market-making agents (Section 1.4.1) with rich order-book state features
Policy gradient (REINFORCE, actor-critic)	A parameterized policy $\pi_\theta(a \mid s)$ directly	Continuous action spaces	Trade-size or quote-skew decisions that vary continuously rather than over a small discrete menu
Proximal Policy Optimization (PPO)	A parameterized policy, with a stabilized update rule	Continuous actions, need for reliable, stable training	Portfolio-rebalancing agents (Section 1.4.2) and production execution systems such as Section 1.5.1's LOXM

Table 1.3: A summary of the reinforcement learning methods introduced in this chapter

action from the state, using the same classification machinery Chapter 6 developed, sidestepping reinforcement learning's exploration problem entirely by never requiring the agent to try an action the expert did not.

Behavioral cloning has a well-known weakness: it is only ever as good as the expert it imitates, and it can fail badly in states the expert rarely visited, since a supervised classifier trained only on an expert's typical behavior has no guidance for how to recover once the agent's own small errors push it into an unfamiliar state the expert's demonstrations never covered, compounding over a long trajectory in a way a single supervised prediction never does.

Inverse reinforcement learning takes a different approach to the same underlying idea. Rather than directly imitating the expert's actions, it infers the reward function the expert appears to be optimizing, then solves the resulting Markov decision process using this chapter's own methods. The resulting policy can, in principle, generalize sensibly to states the expert never actually visited. The cost is a harder and less well-posed estimation problem, since many different reward functions can often rationalize the same observed expert behavior equally well.

Both techniques are directly relevant to the applications developed next: a system like Section 1.5.1's LOXM, reported to be trained on "a large history of past trades together with simulated trading scenarios," plausibly combines this kind of demonstration-based learning, bootstrapping an initial policy from historical human and algorithmic execution decisions, with further trial-and-error refinement in simulation, rather than learning a trading policy from pure, unguided exploration alone.

This hybrid approach is common in practice precisely because it addresses reinforcement learning's sample-inefficiency problem directly: starting from a reasonable, expert-informed policy rather than from nothing gives the trial-and-error refinement stage a far smaller gap to close, the practical advantage that makes imitation-based pretraining worth the additional complexity in a genuinely high-stakes application.

1.4 Financial Applications

This part points the machinery at execution, allocation, and hedging, recovering earlier chapters' answers where they exist.

Wulin.Suo@Queensu.ca

1.4.1 Reinforcement Learning for Optimal Execution and Market Making

Optimal execution was among the earliest documented applications of reinforcement learning in finance. Nevmyvaka, Feng, and Kearns (2006) applied tabular Q-learning, structurally similar to Section 1.3.1's worked example but with a far larger state space built from order-book features, to real historical limit order book data. They found that a learned execution policy could meaningfully reduce trading costs relative to the simple TWAP and VWAP benchmarks Chapter 13's Section 13.3.4 introduced, without ever being given a closed-form market-impact model to optimize against.

The state in a modern version of this problem typically includes remaining inventory, time remaining in the execution window, the current bid-ask spread, and recent order-flow imbalance; the action is how many shares to submit, and at what limit price, this period; and the reward is the negative implementation shortfall (Chapter 13's Section 13.6.4 transaction-cost-analysis measure) realized on that period's fill.

Unlike Almgren-Chriss, which commits in advance to a fixed schedule regardless of how the market subsequently behaves, a reinforcement-learned execution policy can condition its trade size on the order book it actually observes, trading faster when liquidity is unusually deep and slowing down when the book thins out, a genuinely adaptive behavior no closed-form schedule can replicate.

A Worked Example: Q-Learning Recovers the Almgren-Chriss Optimal Trajectory

Section 1.3.1's worked example verified Q-learning against a known answer on a small, three-state problem. It is worth doing the same on a problem closer to genuine trading practice: Chapter 13's own Almgren-Chriss worked example, the 250-share order split across $N = 5$ intervals with impact coefficient $\eta = 0.01$, per-interval volatility $\sigma = 2\%$, and risk aversion $\phi = 0.5$, whose closed-form optimal trajectory left unexecuted shares (250, 194.2, 142.4, 93.4, 46.2, 0) at a total risk-adjusted cost of 139.4.

Reformulated as a Markov decision process, the state at interval j is the inventory x_{j-1} still unexecuted, discretized to the nearest 5 shares (a grid of 51 possible inventory levels), and the action is how many of those shares, again in multiples of 5, to sell this interval.

The reward for selling a_j shares and leaving $x_j = x_{j-1} - a_j$ unexecuted is the negative of Chapter 13's own per-interval cost,

$$r_j = -\left[\eta \frac{a_j^2}{\tau} + \phi\sigma^2\tau\, x_j^2\right],$$

with $\tau = 1$ matching Chapter 13's own normalization, and the last interval ($j = 5$) forces full liquidation, $x_5 = 0$, exactly as the closed-form problem requires. Unlike Section 1.3.1's tree, this environment is entirely **deterministic**: choosing an action leaves no residual uncertainty about the resulting state or reward, since Almgren-Chriss prices timing risk analytically through the variance term $\phi\sigma^2\tau x_j^2$ rather than by exposing the agent to a randomly sampled price path.

Reinforcement learning is still the right tool here for a different reason than in Section 1.3.1: not to cope with stochastic transitions, but to search a state-action space of roughly 5,300 discretized possibilities, far too many to enumerate by hand, without ever being given the closed-form $\sinh(\cdot)$ trajectory formula Chapter 13 derived analytically.

Exact backward-induction dynamic programming over this same discretized grid, computable directly since only 51 inventory levels and 4 real decision points are involved, gives an optimal discretized trajectory of (250, 195, 145, 95, 45, 0) shares at a total cost of 139.52, close to but not identical to the continuous closed-form values, an expected consequence of rounding a continuous optimum onto a 5-share grid rather than any error in the discretization itself.

Tabular Q-learning, trained model-free for 600,000 episodes with an ε-greedy behavior policy decaying from $\varepsilon = 0.5$ to 0.05 and a visit-count-decayed learning rate, **converges to this same trajectory and this same cost**, 139.52, not merely close to it.

This is a meaningfully different outcome from Section 1.3.1's worked example, where the learned "hold" values settled within roughly one percent of their closed-form targets rather than matching them exactly: because this environment is deterministic, every reward Q-learning observes is noise-free, so once training has explored a state-action pair enough times to identify its best continuation, the learned value converges to it exactly, with no residual estimation error left to average away.

Wulin.Suo@Queensu.ca

Figure 1.4 makes the backward-induction computation itself concrete. Dynamic programming does not work through the five intervals in the order a trader would experience them; it starts from the one stage with no real decision left, the forced liquidation at x_4, and solves backward, one stage at a time, using each already-solved stage's value function to solve the one before it, until $V_1(x_0)$ finally gives the answer for the full, five-interval problem.

Only after this entire backward pass is complete can the optimal trajectory be traced forward, choosing at each state whichever action attains the value just computed for it, the bottom row of the figure.

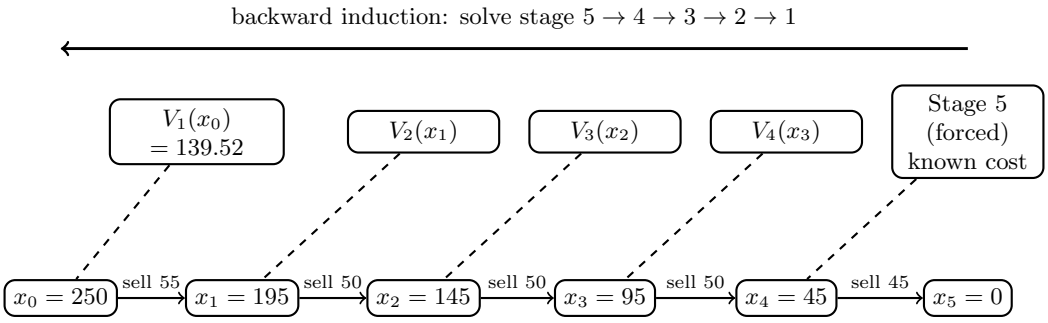


Figure 1.4: Backward-induction dynamic programming for the execution problem

Table 1.4 places the Q-learning-recovered trajectory alongside the uniform TWAP schedule and the fully aggressive single-shot execution Chapter 13 also scored, confirming the same efficient-frontier ordering Chapter 13 established analytically now emerges from an agent that was never given the underlying cost formula at all.

Strategy	Unexecuted-share trajectory	Total cost
Fully aggressive (single shot)	$(250, 0, 0, 0, 0, 0)$	625.00
Uniform TWAP	$(250, 200, 150, 100, 50, 0)$	140.00
Q-learning (learned, discretized)	$(250, 195, 145, 95, 45, 0)$	139.52

Table 1.4: Total risk-adjusted execution cost under three strategies for the same 250-share, $N = 5$-interval order ($\eta = 0.01$, $\sigma = 2\%$, $\phi = 0.5$)

Figure 1.5 plots the same three trajectories, making the front-loading pattern immediately visible: the fully aggressive strategy liquidates instantly, the uniform TWAP schedule is a straight line, and the Q-learning trajectory sits just below TWAP from the first interval on, with fewer shares left unexecuted, trading a little faster than uniform while inventory is largest and tapering to match TWAP by the final interval, exactly the mild front-loading Chapter 13's continuous closed-form solution also produced.

Market making extends naturally to the same framework. Chapter 13's Avellaneda-Stoikov model derives a closed-form optimal quote skew from an assumed order-arrival intensity function and a known risk-aversion parameter.

A reinforcement-learning market maker instead learns a quoting policy directly from simulated or historical order flow, with state features again drawn from current inventory, time-to-close, and order-book depth, action given by the bid and ask quotes (or, in a simplified discrete-action version, a small set of candidate skews), and reward given by realized spread capture net of inventory risk and adverse selection, the same adverse-selection concern Chapter 14's Glosten-Milgrom model formalizes. The appeal is identical to the execution case: a learned policy can adapt to order-arrival patterns that deviate from Avellaneda-Stoikov's assumed functional form, at the cost of requiring a large amount of representative training data and offering none of the closed-form model's transparency about why a given quote was chosen, a tradeoff this chapter's closing sections return to directly.

Wulin.Suo@Queensu.ca

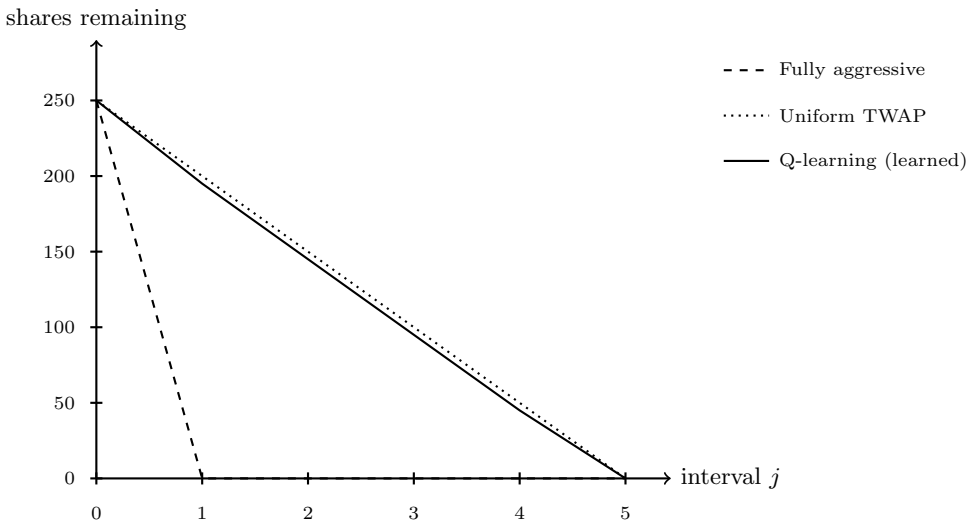


Figure 1.5: Unexecuted-share trajectories under the three strategies of Table 1.4

Designing States, Actions, and Rewards in Practice

Neither problem above specifies its Markov decision process uniquely, and the specific design choices a practitioner makes matter enormously to whether the resulting agent actually learns something useful. A state representation that omits recent order-flow imbalance discards the predictive signal Chapter 14's order-flow-imbalance regression showed carries real information about near-term price movement, weakening the agent below what a well-informed human trader, or a simpler rule-based system that explicitly conditions on that signal, could achieve. A state that includes too many redundant or noisy features, conversely, slows learning by forcing the agent to discover on its own which features actually matter, the feature-engineering discipline Chapter 6's Section 6.5.2 already developed for supervised models, now facing the added difficulty that a reinforcement-learning agent must learn this from its own trial and error rather than from a fixed, already-labeled training set.

Reward design carries the same tension in sharper form, because Section 1.6 shows that an imperfect reward does not merely produce a slightly worse agent, it produces an agent that is often precisely, confidently wrong about what it is optimizing. A common practical compromise is to shape the reward with an intermediate, densely observed signal, a small penalty proportional to inventory held at each step, say, in addition to the sparse, terminal profit-and-loss outcome. This speeds up learning considerably, since the agent gets useful feedback every step rather than only at the end of a long execution episode. But it introduces its own risk: if the shaping term is not carefully scaled relative to the true objective, the agent can learn to over-optimize the intermediate proxy, holding needlessly little inventory, say, at the expense of the actual goal it was meant merely to support learning toward. There is no universal recipe for getting this balance right; in practice it is validated the same way any other model choice in this book is validated, by comparing the resulting policy's realized, out-of-sample performance against the closed-form or rule-based benchmark it is meant to improve on, exactly as Section 1.5.2 recommends.

1.4.2 Reinforcement Learning for Portfolio Management

Chapter 12's mean-variance framework solves a single-period allocation problem: given today's estimated expected returns and covariance matrix, find the weights that optimize a risk-return tradeoff today. Section 12.4's practical-allocation discussion acknowledges that real portfolios are rebalanced repeatedly and that transaction costs make each rebalancing decision depend on the portfolio's current position, but it stops at flagging this complexity rather than solving the resulting multi-period problem directly.

Framed as a Markov decision process, the state is the portfolio's current holdings together with estimated risk-factor exposures (Chapter 12's Section 12.3.3 multi-factor model is a natural source of these features);

Wulin.Suo@Queensu.ca

the action is a new target weight vector, or equivalently the trades needed to reach it. The reward combines the period's realized return with an explicit penalty for transaction costs and, if desired, for realized volatility or drawdown, letting the agent learn a rebalancing policy that trades off today's allocation quality against the cost of getting there, rather than treating every rebalancing decision as a fresh, myopic optimization the way Chapter 12 does.

This framing is genuinely more general than Chapter 12's static optimization, and that generality is exactly its cost. A reinforcement-learning portfolio agent requires either a large amount of representative historical data or a realistic market simulator to train in, since a poorly specified reward (one that fails to penalize drawdown, say, or that rewards raw return without any risk adjustment) will produce a policy that optimizes the wrong objective with no warning that anything has gone wrong, a version of the "reward hacking" problem Section 1.6 discusses in more depth.

In practice, most institutional applications of reinforcement learning to portfolio management remain closer to research than to production, used to explore whether a learned, adaptive rebalancing policy can outperform Chapter 12's static mean-variance benchmark net of realistic transaction costs, rather than to replace that benchmark outright. The honest current state of the field is that reinforcement learning has demonstrated clearer, more reproducible gains in the execution and market-making applications of Section 1.4.1, where the reward (realized trading cost) is comparatively unambiguous to specify, than in full portfolio management, where the "right" reward is itself a genuinely open question tied to a client's actual, often unstated, risk preferences.

Multi-period portfolio choice has a classical closed-form precedent worth naming, since it clarifies exactly what a learned policy would need to rediscover on its own. Chapter 1's Section 1.5.1 introduced the **Kelly criterion** for a single repeated bet; the same result, maximizing the expected *logarithm* of long-run wealth rather than expected wealth itself, extends naturally to repeated portfolio rebalancing under a power-utility objective, producing an allocation fraction that avoids both the ruin risk of over-betting and the excessive caution of under-betting. This is itself a sequential decision problem with a known closed-form answer under simplifying assumptions, constant, known return and variance parameters, the "Chapter 12 could already have gone further" observation that opened Section 1.2.2. It gives a useful sanity check for a learned rebalancing agent: on a simplified simulated market where the Kelly fraction is known exactly, a correctly implemented reinforcement-learning portfolio agent should converge toward that same fraction, an internal-consistency check with the same purpose as Section 1.3.1's comparison against Chapter 5's closed-form option value, before that agent is ever trusted on a real, non-simplified market where no such closed form is available to check against.

A Worked Example: Why Model-Free Search Struggles Here

Running that sanity check honestly, rather than only asserting it would work, is instructive precisely because it does not go as cleanly as Sections 1.3.1 and 1.4.1. Chapter 1's repeated bet, an even-money wager ($b = 1$) won with probability $p = 0.6$, has closed-form optimal fraction $f^* = p - q/b = 0.20$ and long-run growth rate $g(f^*) \approx 0.0201$. Framed as a Markov decision process, this is a single, repeated state (the optimal fraction does not depend on current wealth, so no wealth variable needs to be tracked) with action f discretized to a grid of 5% increments, $f \in \{0.00, 0.05, \ldots, 0.95\}$, and an immediate, noisy reward $r = \ln(1 + f)$ with probability p or $r = \ln(1 - f)$ with probability q, drawn independently every period, no bootstrapping required since there is no next state to look ahead to. In other words this is a multi-armed bandit, structurally identical to Section 1.2.3's venue-routing problem with twenty arms in place of three. What it does not share with that problem is any hope of an easy answer, and the reason is worth stating before the numbers arrive.

What makes this problem genuinely hard for a model-free method is the relationship between the reward's noise and the size of what there is to learn: the gap between the optimal action's growth rate and its nearest neighbors on the grid, $g(0.20) - g(0.15) \approx 0.0013$ and $g(0.20) - g(0.25) \approx 0.0013$, is tiny, while a single bet's outcome swings between $\ln(1.20) \approx 0.182$ and $\ln(0.80) \approx -0.223$, a standard deviation roughly 150 times larger than the gap Q-learning needs to resolve.

Trained for 200,000 simulated bets with the same ε-greedy, visit-count-decayed setup used throughout this chapter, tabular Q-learning settles on $f = 0.15$, not $f^* = 0.20$, leaving $g(0.20) - g(0.15) \approx 0.0013$ of achievable growth rate on the table, about 6.4% of the optimal rate. This is a real, if modest, miss, not the exact or near-exact recovery Sections 1.3.1 and 1.4.1 demonstrated. Figure 1.6 shows why: the growth-rate

Wulin.Suo@Queensu.ca

curve is nearly flat across the entire 0.10-to-0.30 range, so the two marked points, the true optimum and where Q-learning actually converges, sit almost on top of each other.

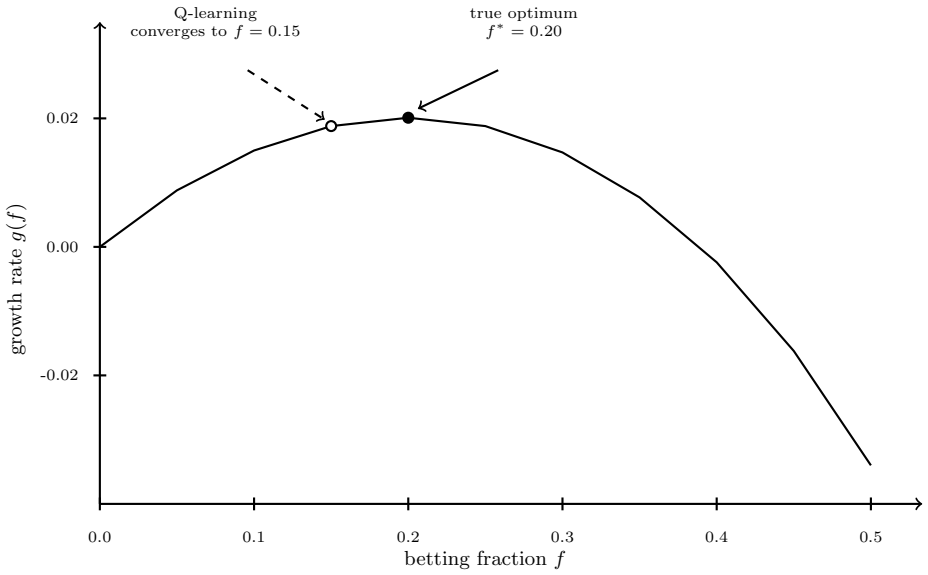


Figure 1.6: The Kelly growth-rate curve $g(f) = p\ln(1+f) + q\ln(1-f)$ for $p = 0.6$, $b = 1$

The contrast worth drawing out is what a **model-based** approach, Section 1.2.5's other category, does with the identical 200,000 observations. Rather than treating each bet's outcome as an opaque reward to average over a large discretized action grid, it exploits the fact that this particular problem has a compact underlying model: a single unknown parameter, the win probability p, plugged directly into Chapter 1's closed form.

Simply counting wins and losses across the same 200,000 simulated bets gives an observed win frequency $\hat{p} = 0.6003$, and substituting into $\hat{f}^* = \hat{p} - (1-\hat{p})/b$ gives $\hat{f}^* \approx 0.2006$, accurate to within 0.0006 of the true optimum, an order of magnitude closer than Q-learning's answer from the exact same data. This is Section 1.2.5's model-based-versus-model-free distinction made numerically concrete rather than only asserted, and it sharpens Section 1.2.2's opening argument: reinforcement learning is the right tool once no compact model is available, but when one is, as it is here, a single Bernoulli parameter standing in for the entire problem, exploiting that structure directly can be dramatically more sample-efficient than a black-box search over a discretized action grid, the gap this worked example just measured.

Not every sequential decision problem that can be written as a Markov decision process is equally well suited to being solved by treating it as one; recognizing when a problem's structure should be modeled directly rather than learned model-free is itself part of the practitioner's job, not something reinforcement learning relieves them of.

1.4.3 Deep Hedging: Learning to Hedge Under Frictions

Chapter 5's replication argument prices an option by continuously holding exactly Δ shares of the underlying, and Chapter 8's delta-normal example measures the risk of an option book through that same delta. Both rest on an assumption no trading desk actually enjoys: that the hedge can be rebalanced continuously and for free. A real desk pays a bid-ask spread and fees on every rebalancing trade, so it faces a genuine dilemma the frictionless theory simply defines away. Rebalance after every tick and transaction costs devour the option premium; rebalance rarely and the position runs unhedged in between, exposed to the gap risk Chapter 8 quantified. Deciding *when* and *by how much* to adjust the hedge, trade by trade, as the underlying moves and expiry approaches, is a sequential decision problem under uncertainty, which is to say it is this chapter's subject.

Wulin.Suo@Queensu.ca

Cast in Section 1.2.1's terms, the state is what the hedger observes, time to maturity τ, the underlying price S_t, and the position h_t currently held; the action is the new position h_{t+1} to hold until the next decision point; and the objective is defined on the terminal outcome: let W_T be the hedger's final wealth after the option is settled and the hedge unwound, premium, trading gains, and all transaction costs included. **Deep hedging** chooses the policy to minimize a convex risk measure of the shortfall,

$$\min_{\pi} \ \rho(-W_T^{\pi}), \qquad \text{for example } \rho = \mathrm{ES}_{95\%},$$

where the risk measure is the Expected Shortfall Chapter 8 introduced, now serving not to measure a fixed position's risk but as the training objective a policy is optimized against.

A small worked example makes the trade-off concrete, and ties it back to numbers this book has already computed. Take Chapter 8's market maker, short the one-year \$50-strike call priced in Chapter 5 ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $C_0 \approx \$6.88$, $\Delta_0 = 0.6115$), and have them hedge it over the option's life on 10,000 simulated price paths (geometric Brownian motion with real-world drift $\mu = 8\%$, 252 daily steps, seed 7 in the companion notebook), paying a proportional transaction cost of 50 basis points of traded value on every trade, the kind of friction an illiquid underlying imposes. The policy class is deliberately simple, a one-parameter family this chapter's earlier examples have prepared: rebalance to the current Black-Scholes-Merton delta only when the held position has drifted more than a *band* b away from it, so $b = 0$ is the textbook daily delta hedge and larger b trades less often. The very first trade is hand-checkable: the hedger buys $\Delta_0 = 0.6115$ shares at \$50, paying $0.005 \times 0.6115 \times \$50 \approx \$0.15$ in costs. Table 1.5 reports, across the 10,000 paths, the mean terminal profit and loss, its standard deviation, the mean total transaction cost, and the 95% Expected Shortfall of the terminal loss.

Policy	Mean P&L	Std. dev.	Mean cost	$\mathrm{ES}_{95\%}$
No hedge	−\$1.32	\$12.54	\$0.00	\$39.30
Daily delta ($b = 0$)	−\$1.60	\$0.59	\$1.56	\$3.01
Band $b = 0.02$	−\$1.37	\$0.61	\$1.34	\$2.82
Band $b = 0.06$	−\$0.95	\$0.70	\$0.93	\$2.56
Band $b = 0.20$	−\$0.60	\$1.92	\$0.53	\$5.76

Table 1.5: Hedging a short one-year \$50-strike call under 50-basis-point proportional transaction costs, across 10,000 simulated paths. The band policy with $b = 0.06$, found by searching the band grid for the lowest $\mathrm{ES}_{95\%}$, beats the textbook daily delta hedge on mean cost and tail risk simultaneously.

The table repays a careful read. Leaving the position unhedged is catastrophic, a \$12.54 standard deviation and a \$39.30 Expected Shortfall against a \$6.88 premium collected. The textbook daily delta hedge crushes that risk twenty-fold, but at a price the frictionless theory never mentions: \$1.56 of expected transaction costs, nearly a quarter of the premium. The band policy at $b = 0.06$, selected by searching the grid for the lowest $\mathrm{ES}_{95\%}$, is the striking row: it cuts mean costs by 40% *and* carries less tail risk than hedging every day, because each rebalance it skips saves a cost that would otherwise be paid with certainty, while the extra hedging error it accepts is small and diversifies across the path. Push the band too wide, though, and unhedged risk floods back in, as the $b = 0.20$ row shows. And the friction level, not taste, dictates the answer: rerun with a 5-basis-point cost and the ES-optimal band shrinks to 0.01, essentially the textbook hedge, confirming that Black-Scholes-Merton delta hedging is not wrong but is the frictionless limit of a more general policy the data-driven search recovers.

Deep hedging proper, introduced by Buehler, Gonon, Teichmann, and Wood in 2019, replaces this one-parameter band with a neural network policy $\pi_\theta(\tau, S_t, h_t)$ trained by stochastic gradient descent over simulated paths to minimize the same kind of convex risk measure, the step from a hand-designed policy family to a learned one that Section 1.3.2 already made for value functions. The gain is generality rather than magic. The network is never shown a delta, a gamma, or any other Greek, yet in the frictionless limit it rediscovers the Black-Scholes-Merton hedge on its own, and away from that limit it can absorb features no closed-form hedge accommodates gracefully: several hedging instruments at once, transaction costs of

Wulin.Suo@Queensu.ca

realistic shape, discrete trading times, position limits, and payoffs with no replicating formula at all. Reports of production use at major derivatives desks followed within a few years of the paper, making hedging, alongside Section 1.5.1's optimal execution, one of the two applications in this chapter where learned policies have genuinely reached practice. The caveats are the ones this chapter has already developed, and they bind here with full force: every path the policy learns from comes out of a simulator, so the quality of the learned hedge is bounded by the realism of that simulator, exactly Section 1.5.2's sim-to-real gap, and a neural hedging policy inherits the governance and explainability burden Chapter 18 attaches to any model whose decisions cannot be read off a formula.

1.4.4 Multi-Agent Dynamics and the Limits of a Single-Agent Framework

Every Markov decision process formalized in Section 1.2.1 assumes a single agent facing a fixed, if stochastic, environment, an assumption real financial markets violate in an important way: the "environment" a trading or market-making agent faces is itself made up, in large part, of other agents, some human, increasingly many themselves algorithmic, each adapting to the same price and order-flow signals the agent under study is adapting to. Chapter 14's hot-potato effect during the 2010 Flash Crash is a vivid, already-documented illustration of this dynamic: individually reasonable inventory-management rules, followed by many participants simultaneously, produced a collectively destabilizing outcome that no single participant's rule, viewed on its own, would have predicted. A reinforcement-learning agent trained against a static, historical simulator implicitly assumes other participants' behavior stays fixed, an assumption that becomes less accurate the larger and more influential the trained agent's own trading becomes, since a strategy profitable at small scale can behave very differently once it is large enough to move the same order flow it was trained to react to.

Multi-agent reinforcement learning extends the single-agent framework of this chapter to model several learning agents simultaneously, each adapting to the others' evolving behavior, and is an active area of research specifically for the market-making and execution problems Section 1.4.1 developed, since real limit order books are genuinely populated by multiple competing, adapting participants. The honest state of this research, as of this book's writing, is that multi-agent methods are considerably harder to train reliably than their single-agent counterparts, convergence guarantees that hold for tabular Q-learning in Section 1.3.1's simple setting generally do not extend to a setting where the environment itself is changing because other agents are learning too. Most production trading systems, including Section 1.5.1's LOXM, are best understood as single-agent policies trained against a historical or simulated environment rather than as genuine multi-agent systems, with the gap between that simplifying assumption and the true, multi-participant market being the sim-to-real and non-stationarity concerns Section 1.6 raises.

A useful, more tractable middle ground treats other participants' aggregate behavior as part of the state rather than as separate learning agents to be modeled explicitly. A single-agent method from Sections 1.3.1 through 1.3.3 can then respond to features like recent order-flow imbalance or realized volatility that summarize what other participants are collectively doing, without attempting the considerably harder problem of modeling each of them individually. That pragmatic compromise is what most of this chapter's real applications, execution and market making among them, actually rely on in practice.

1.5 From Simulation to Deployment

A policy trained in simulation has proved nothing yet, and this part covers what would.

1.5.1 Case Vignette: JPMorgan's LOXM

In 2017, JPMorgan disclosed LOXM, an equity trade execution system trained using reinforcement learning on a large history of past trades together with simulated trading scenarios, initially deployed on the bank's European cash equities desk.

Structurally, LOXM addresses the problem formalized in Section 1.4.1. At each point during an order's execution, the system decides how much of the remaining order to place, at what price, and for how long to wait before reassessing. It balances the risk of trading too aggressively, incurring unnecessary market impact, against the risk of trading too passively, missing the current opportunity and having to execute

Wulin.Suo@Queensu.ca

later at a worse price. That is the same impact-versus-timing-risk tradeoff Almgren-Chriss solves in closed form in Chapter 13, but learned here directly from data rather than derived from an assumed cost function. The system was reported to have been trained on millions of historical and simulated trading scenarios specifically so that it could adapt its behavior to prevailing market conditions rather than following one fixed schedule regardless of context, the central advantage Section 1.4.1 attributes to a learned execution policy over Almgren-Chriss's closed-form alternative.

LOXM is a useful case precisely because it illustrates both sides of this chapter's central tension in a single, real deployment. On one hand, it demonstrates that reinforcement learning for execution is not merely an academic exercise, a major bank judged a learned policy reliable enough to deploy on live client order flow, the kind of practical validation the closed-form models developed earlier in this book have had for decades and that any newer machine-learning-based alternative must eventually earn. On the other hand, a deep reinforcement-learning execution policy is far harder to explain to a regulator or an internal model validator than an Almgren-Chriss schedule, whose entire logic reduces to a handful of interpretable parameters, a governance burden Section 1.6 and Chapter 18's explainability toolkit both address directly, and one that any institution deploying a system like LOXM must take on explicitly rather than treat as an afterthought.

LOXM was also an early, rather than an isolated, example. In the years since 2017, industry reporting has described similar reinforcement-learning-based execution research and deployments at several other major banks and trading firms. This is consistent with this chapter's broader claim that optimal execution, alongside Section 1.4.3's hedging application, has the clearest track record of reinforcement learning actually reaching production, precisely because its reward, realized execution cost relative to a well-defined benchmark, is comparatively unambiguous to specify correctly, the property Section 1.4.2 identified as still generally missing from full portfolio management.

1.5.2 Simulation, Backtesting, and Off-Policy Evaluation

Every reinforcement learning agent in this chapter is trained by trial and error, which raises an operational question none of this book's earlier, purely supervised models faced: where does the trial and error actually happen? Training directly on live markets is rarely acceptable, since an untrained or partially trained policy will, by construction, take some genuinely bad actions while exploring, and a bad trading action costs real money in a way a bad prediction from a still-training classifier does not. The standard alternative is to train in a **simulator**, a model of the market environment built from historical data, that can be reset and replayed as many times as needed without financial consequence; Chapter 13's own backtesting framework (Section 13.6) is a close cousin of this idea. It inherits the same core limitation, a simulator built from historical data reflects how the market behaved historically, including in response to other participants' historical order flow, not necessarily how it would respond to a new agent's own actions once deployed at scale, a gap sometimes called the **sim-to-real gap** that is at least as serious a concern for a reinforcement-learning trading agent as ordinary backtest overfitting is for the supervised models Chapter 13 already warned about.

A related and more subtle problem is **off-policy evaluation**: given historical data generated by one policy (a human trader's decisions, say, or an older execution algorithm), how good would a different, newly trained policy have performed, without ever actually running it? This matters because historical market data reflects only the actions the deployed policy actually took, not the counterfactual outcomes of actions it did not take. That is the missing-counterfactual problem Chapter 18's algorithmic-recourse discussion raised for credit decisions, now recurring in a sequential setting where a single wrong counterfactual estimate can compound over many steps. Techniques such as importance sampling reweight historical outcomes to estimate what a new policy would have achieved, but their reliability degrades quickly the more the new policy's behavior diverges from the one that generated the historical data. That is why a genuinely novel learned policy is typically rolled out cautiously: first in simulation, then on a small fraction of live order flow, with continuous monitoring for the kind of performance drift Chapter 15's precision-drift and Chapter 18's population-stability-index discussions already developed for other models. It is not deployed at full scale on the strength of a backtest or an off-policy estimate alone.

This staged approach, simulator, then a small live allocation, then a gradually increasing one, is sometimes called **curriculum deployment**, and it mirrors a discipline this book has already recommended elsewhere for a different reason: Chapter 8's three-lines-of-defense model and Chapter 18's staged-deployment recom-

Wulin.Suo@Queensu.ca

mendation for high-stakes machine learning models both argue for this kind of graduated rollout, independent of whether the model in question is a reinforcement-learning policy or a static classifier. What is genuinely new to the reinforcement-learning setting is that the policy itself may continue to update during this staged rollout: a live, still-learning system rather than a fixed, already-frozen model. The monitoring in place must therefore detect not only a fixed policy's performance drifting away from its backtested expectation, but a still-adapting policy drifting toward behavior nobody has explicitly validated at all. That is a strictly harder monitoring problem than the ones Chapters 15 and 18 originally posed.

1.5.3 Evaluating Trained Policies: Metrics and Variance Across Training Runs

Chapter 13's Section 13.6.1 developed a standard toolkit, Sharpe ratio, maximum drawdown, hit rate, for evaluating a trading strategy's backtested performance, and every one of those metrics applies unchanged to a reinforcement-learning policy's simulated or live performance. What reinforcement learning adds is a genuinely new source of variability that a closed-form model like Almgren-Chriss simply does not have: since every method in this chapter is trained via a stochastic process, random initialization, ε-greedy exploration, and, for the neural-network methods of Sections 1.3.2 and 1.3.3, stochastic gradient descent itself, two training runs of the identical algorithm on identical data, differing only in their random seed, can converge to meaningfully different policies with meaningfully different realized performance.

Section 1.3.1's worked example makes this concrete in miniature. It was trained with a fixed seed (seed = 7) specifically so that its reported numbers, \$2.5328 and \$4.9234 among them, would be exactly reproducible for a reader rerunning the companion notebook. A different seed would converge to slightly different learned values, still close to the closed-form \$2.5641 and \$4.8718, since the underlying algorithm and its guarantees do not depend on the seed, but not numerically identical to the values reported in Table 1.2. In a small, three-state problem with a known correct answer to check against, this seed sensitivity is a minor curiosity; in a large, real trading application with no closed-form answer available, it is a serious practical concern, since a single training run's backtested performance could simply be a favorable draw rather than a property of the algorithm the institution intends to rely on.

The standard remedy is to train multiple independent runs from different seeds and report performance as a distribution rather than a single number. It is standard in machine learning generally, but has not been needed anywhere else in this book, whose supervised and closed-form models are deterministic given their input data. It is the statistical caution Chapter 6's confidence intervals and Chapter 7's out-of-sample RMSE comparisons already applied to estimation uncertainty, now applied to the additional uncertainty introduced by the training process itself, not just by the data.

1.5.4 Practical Considerations: What Deployment Actually Requires

The methods and applications developed above can leave an unrealistic impression that adopting reinforcement learning is primarily an algorithmic choice, DQN or PPO, Q-learning or actor-critic, when in practice the harder and more expensive decisions are organizational and infrastructural, and worth naming explicitly before this chapter closes.

A trustworthy simulator is usually the binding constraint, not the algorithm. Every method in this chapter needs a large volume of training experience, and Section 1.5.2 already established why that experience typically cannot come from live markets. Building a market simulator accurate enough to train a genuinely useful policy, one that reproduces realistic order-book dynamics, price impact, and other participants' plausible reactions, is a substantial engineering undertaking in its own right, often larger than the effort spent on the learning algorithm itself. Few institutions can justify that project unless the underlying decision problem is important and recurring enough to be worth the fixed investment. That is why optimal execution at a large bank, a genuinely high-volume, high-value recurring decision, has been an early adopter while more bespoke, lower-frequency problems largely have not.

The required skill set differs from, and must complement rather than replace, this book's earlier chapters. A team building a reinforcement-learning trading system needs the market-microstructure fluency Chapters 13 and 14 developed, the statistical and machine-learning foundations of Chapter 6, and, on top of both, genuine expertise in the distinct engineering discipline of reinforcement learning itself: stable training, careful hyperparameter selection, and the evaluation methodology of Section 1.5.2. That

Wulin.Suo@Queensu.ca

combination is still comparatively rare, and most institutions build it gradually rather than hire for all of it at once. This is one more reason closed-form models like Almgren-Chriss and Avellaneda-Stoikov remain the right default choice for most applications: they require the market-microstructure expertise this book has developed throughout, but not the additional, specialized reinforcement-learning engineering capability on top of it.

The honest cost-benefit comparison is against the closed-form alternative, not against doing nothing. A closed-form model, once specified, costs almost nothing to run and is immediately auditable by the model-risk-management process Chapter 8 established; a reinforcement-learning policy costs substantially more to build, validate, and monitor on an ongoing basis, and that additional cost is only justified when the closed-form model's simplifying assumptions are demonstrably costing real money, the market-impact and reward-specification gaps Section 1.2.2 identified. A useful discipline, consistent with this book's compute-first approach throughout, is to build and validate the closed-form baseline first, know precisely what it costs in realized performance, and only then invest in a learned alternative with a clear, pre-specified bar for how much improvement would justify the added complexity and ongoing governance burden.

Compute cost is a real, ongoing line item, not a one-time expense. Training a deep Q-network or a PPO policy over the hundreds of thousands or millions of simulated episodes a realistic problem requires consumes meaningfully more computing resources than fitting any single supervised model elsewhere in this book. That cost recurs every time the policy is retrained, whether on a fixed schedule to keep pace with the non-stationarity Section 1.6 describes, or in response to a detected drift in live performance. This ongoing retraining cost, together with the ongoing monitoring cost Section 1.5.2 requires, belongs in the same total-cost-of-ownership comparison as the upfront simulator investment discussed above, not treated as a one-time project expense that ends once a policy is first deployed.

1.6 Challenges in Reinforcement Learning for Finance

Reinforcement learning inherits every model-risk concern raised elsewhere in this book and adds several that are specific to its sequential, trial-and-error structure.

Reward specification is the entire problem in disguise. An agent optimizes the reward function it is given, not the objective its designer actually had in mind, and the gap between the two is usually invisible until the agent finds and exploits it, a failure mode generally called **reward hacking**. An execution agent rewarded purely for beating a benchmark price, with no penalty for market impact left behind for the next order, can learn to trade in ways that look good on that one metric while quietly making the market worse for the desk's future orders; a portfolio agent rewarded for raw return with no risk adjustment can learn to take on the tail risk a real client would never accept. Specifying a reward that genuinely captures an institution's objective, including the parts that are hard to quantify, is consistently harder in practice than designing the learning algorithm itself.

Non-stationarity compounds ordinary concept drift. Chapter 15 and Chapter 18 both warned that a supervised model's accuracy can degrade as the world changes. A reinforcement-learning trading agent faces the same risk with an added twist, since other market participants adapt to *its* behavior too, particularly once a learned strategy is large enough to influence prices. The environment itself becomes something that changes partly *because of* the agent's own past actions, a genuinely adversarial dynamic no supervised classifier in this book has had to contend with.

Reinforcement learning is dramatically more sample-hungry than the supervised methods elsewhere in this book. Section 1.3.1's worked example needed 100,000 simulated episodes to estimate just six numbers to within roughly one percent, and that problem has only three states and two actions. Chapter 6's supervised classifiers, by contrast, routinely reach useful accuracy from a training set several orders of magnitude smaller relative to their problem's apparent complexity, because a supervised label provides direct, unambiguous feedback on every single example, while a reinforcement-learning reward, especially a delayed, terminal one, provides comparatively little information per episode about which of many earlier actions actually deserves credit for it, the credit-assignment difficulty this section returns to below. This sample inefficiency is precisely why Section 1.5.4's simulator investment is not optional infrastructure but the central prerequisite for any of this chapter's methods to work at all outside of small, hand-verifiable settings like this chapter's own worked example.

Wulin.Suo@Queensu.ca

Exploration is expensive and risky in a way it is not in a game. The ε-greedy exploration in Section 1.3.1's worked example costs nothing beyond simulated time; the same exploration on a live trading desk means deliberately taking some actions believed to be suboptimal, in real markets, with real capital, specifically to learn whether they might be better than currently believed. This is a fundamentally different risk profile from a self-driving research agent exploring a video game, and it is the central reason simulation-based and off-policy training, Section 1.5.2's subject, are not merely convenient but close to mandatory for any serious financial application.

Explainability and governance do not disappear just because a policy performs well. A deep reinforcement-learning policy inherits all the opacity concerns Chapter 18 raised for deep neural networks generally, compounded by the fact that its behavior is defined over an entire trajectory of decisions rather than a single prediction, making a SHAP-style, single-decision explanation only a partial account of what the policy is actually doing. A model validator or regulator evaluating a system like Section 1.5.1's LOXM needs some account of why the policy behaves as it does across a representative range of market conditions, not just a performance summary, the model-risk-management discipline Chapter 8 established for market-risk models and Chapter 18 extended to explainability generally, now applied to a policy that acts rather than merely predicts.

Credit assignment gets harder the longer the horizon. A supervised classifier receives immediate, unambiguous feedback on every prediction it makes, Chapter 6's confusion matrix scores a decision the moment it is made. A reinforcement-learning agent, by contrast, often does not learn whether an early action was good or bad until many steps later, when the resulting reward finally arrives, the **credit assignment problem**. Section 1.3.1's worked example sidesteps this almost entirely, since its longest episode is only two decisions deep, but a full trading day's worth of execution decisions, or a multi-quarter portfolio-rebalancing episode, can make it genuinely difficult to tell which of many earlier actions actually caused a later gain or loss, slowing learning and demanding far more training data than a supervised problem of comparable apparent complexity.

Simulators are themselves models, with their own model risk. Section 1.5.2's sim-to-real gap is not merely a data-availability inconvenience. A market simulator built from historical data encodes its own assumptions about how prices and order flow behave, and an agent trained exclusively against a flawed simulator will learn a policy that is optimal for that simulator's biases rather than for the real market. That is no different in kind from Chapter 8's warning that a risk model is only as trustworthy as the assumptions behind it. Validating a trading simulator against out-of-sample market behavior, not just validating the agent trained inside it, is therefore a necessary part of the same model-risk-management discipline this chapter has invoked throughout.

The regulatory and legal framework for autonomous trading policies is still catching up. Chapter 8's model-risk-management guidance and Chapter 18's explainability requirements were largely written with static, single-prediction models in mind. A policy that adapts its own behavior over time, potentially continuing to learn after deployment, raises validation questions those frameworks do not yet answer: how often must a live, still-adapting policy be re-approved, and what constitutes an adequate explanation of a policy rather than a prediction? Existing supervisory guidance does not address either with the specificity it applies to a fixed logistic regression or gradient-boosted tree. Institutions deploying reinforcement learning in a regulated trading or advisory context should expect to extend, and in places invent, their own governance answers rather than assume existing frameworks already cover this case completely.

1.7 Key Takeaways

Reinforcement learning formalizes sequential decision-making as a Markov decision process, with states, actions, rewards, and a discount factor, and its objective, maximizing expected discounted return, generalizes the one-shot optimizations this book has relied on since Chapter 5.

The Bellman equation expresses a state's or action's value recursively in terms of immediate reward plus discounted continuation value, and Chapter 5's binomial-tree backward induction already solves a Bellman optimality equation by hand. This chapter's central worked example made that fact exact, reformulating the same tree as a three-state MDP and recovering its known value, \$2.5641, using model-free tabular Q-learning instead. The learned and closed-form solutions agree to within roughly one percent after 100,000 simulated

Wulin.Suo@Queensu.ca

episodes.

Deep Q-networks and policy-gradient methods, including actor-critic algorithms and PPO, extend the same value-based and policy-based logic respectively to large or continuous state and action spaces where a table is impossible to maintain.

Optimal execution and market making, Chapter 13's closed-form models, generalize naturally into reinforcement-learning problems once market impact and order-arrival behavior are too complex for a clean parametric form. That generalization has already been validated in research (Nevmyvaka, Feng, and Kearns, 2006) and in industry practice (JPMorgan's LOXM). Portfolio management admits the same framing but remains a harder, more open problem, chiefly because its reward is harder to specify unambiguously than an execution cost.

Every one of these applications inherits reinforcement learning's distinctive risks: reward misspecification, non-stationarity driven by other agents' adaptation, the expense and danger of live exploration, severe sample inefficiency relative to this book's earlier supervised methods, and an explainability burden that compounds Chapter 18's concerns rather than resolving them. All of these argue for simulation-based training, cautious staged deployment, and continuous monitoring, rather than treating a well-trained policy as a finished, self-sufficient product.

Markets are also not single-agent environments: other participants adapt to a learned policy's own behavior just as it adapts to theirs. That is why most production systems remain single-agent approximations of a genuinely multi-agent problem. It is also why the practical bottleneck to adopting any of this chapter's methods is usually a trustworthy simulator and the specialized engineering skill to train against it reliably, not the choice of algorithm itself.

1.8 Suggested Exercises

1. Using Section 1.3.1's three-state Markov decision process, verify by hand that $Q^*(S_d, \text{hold}) = \gamma[q(0) + (1-q)(9.50)] \approx \$4.8718$, and explain in your own words why immediate exercise is optimal at that state despite continuation having a positive expected value.

2. Suppose the down-move factor in Section 1.3.1's tree were $d = 0.8$ instead of 0.9, with $u = 1.2$ and $R = 1.04$ unchanged. Recompute q, the terminal payoff at the down-down node, and the closed-form value $V^*(S_0)$, and state whether the optimal policy at S_0 changes.

3. Explain why a discount factor of $\gamma = 1$ would be inappropriate for an infinite-horizon trading agent that earns a small positive expected reward every period, and describe what specifically goes wrong with the return G_t in that case.

4. Using this chapter's Q-learning update rule, compute by hand the new value of $Q(S_0, \text{hold})$ after a single simulated transition from S_0 to S_d with reward 0, given a current table of $Q(S_0, \text{hold}) = 2.00$, $Q(S_d, \text{exercise}) = 5.00$, $Q(S_d, \text{hold}) = 4.50$, a learning rate $\alpha = 0.1$, and $\gamma \approx 0.9615$.

5. Compare Almgren-Chriss (Chapter 13, Section 13.3.5) and a reinforcement-learning execution agent (Section 1.4.1) along three dimensions: the data required to use each, the interpretability of the resulting trading schedule, and each approach's ability to adapt mid-execution to an order book that behaves differently than assumed.

6. Describe a reward function for a portfolio-rebalancing agent (Section 1.4.2) that would encourage the exact kind of reward hacking Section 1.6 warns about, and propose a specific, concrete change to that reward function that would discourage it.

7. Explain why off-policy evaluation (Section 1.5.2) becomes less reliable the more a newly trained policy's behavior diverges from the policy that generated the available historical data, and describe one practical consequence this has for how a bank should stage the rollout of a new trading agent.

8. A market-making desk considers replacing its Avellaneda-Stoikov quoting model (Chapter 13, Section 13.4) with a learned reinforcement-learning policy. List three pieces of evidence a model validator should require before approving this replacement, drawing on both this chapter's discussion and Chapter 8's model-risk-management framework.

Wulin.Suo@Queensu.ca

9. Explain, in your own words, why maximizing expected log-wealth (the Kelly criterion, Section 1.4.2) produces a different, generally more conservative allocation than maximizing expected wealth directly, and describe why this makes the Kelly fraction a useful sanity check for a reinforcement-learning portfolio agent trained on a simplified simulated market.

10. Using Section 1.4.4's discussion, explain why a reinforcement-learning execution agent trained against a fixed historical simulator might perform worse than expected once deployed at large enough scale to influence the very order flow it was trained on, and describe one practical step a firm could take to detect this problem before it causes significant losses.

11. Explain why behavioral cloning (Section 1.3.4) can fail badly in a state the expert trader rarely visited, even though it performs well on states similar to the expert's typical behavior, and describe how inverse reinforcement learning's approach, inferring a reward function rather than directly imitating actions, is intended to address this specific weakness.

12. Section 1.2.3's desk has sent four orders to venue A, giving $Q(A) = 1.60$, and six to venue B, giving $Q(B) = 1.85$. (a) A fifth order to A fills at 3.00 basis points; apply the running-average rule to compute the updated $Q(A)$, and state which venue the greedy policy now selects. (b) A single fill has a standard deviation of 2.0 basis points. Compute the standard error of each of the two estimates and explain why the switch in part (a) is far weaker evidence than it looks. (c) A fourth venue D, whose true mean of 2.4 basis points beats every existing venue, is now connected, and the desk keeps $\varepsilon = 0.2$ with exploration spread uniformly over all four. Compute the new permanent shortfall $\varepsilon(\mu^* - \bar{\mu})$, compare it to the three-venue figure, and explain why adding a venue better than all the others nonetheless raises the per-order cost of a fixed exploration rate.

13. **Challenge (real data).** Download at least three years of daily prices for a real, liquid stock (via `yfinance`). (a) Discretize the state space into a small number of buckets based on the sign and magnitude of the trailing five-day return, in the spirit of Section 1.3.1's three-state example, and train a tabular Q-learning agent with actions {buy, hold, sell} and a reward equal to the next day's realized return (long) or its negative (short), using this chapter's Q-learning update rule. (b) Compare the trained agent's cumulative return over the full sample to a simple buy-and-hold benchmark, then re-run training on only the first half of the sample and evaluate the resulting policy on the second half, and report whether the agent's apparent advantage survives this split. (c) Using Section 1.5.2's discussion of off-policy evaluation, explain why backtesting a single historical price path is an especially weak form of policy evaluation for a trading agent, and describe one practical safeguard, drawing on Section 1.4.4 or Chapter 8's model-risk-management framework, a firm should require before deploying such an agent with real capital.

1.9 Further Reading

- Sutton, R. S., and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, 2018. The standard textbook treatment of the Markov decision process framework, the Bellman equation, and value-based and policy-based methods developed throughout this chapter.

- Mnih, V., et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, vol. 518, 2015, pp. 529–533. The original Deep Q-Network paper underlying Section 1.3.2's function-approximation extension of tabular Q-learning.

- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv*, 2017 (arXiv:1707.06347). The algorithm underlying the modern policy-gradient methods discussed in Section 1.3.3.

- Nevmyvaka, Y., Y. Feng, and M. Kearns, "Reinforcement Learning for Optimized Trade Execution," *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 673–680. An early and influential application of tabular Q-learning to real limit-order-book execution data, underlying Section 1.4.1's discussion.

Wulin.Suo@Queensu.ca

- Almgren, R., and N. Chriss, "Optimal Execution of Portfolio Transactions," *Journal of Risk*, vol. 3, no. 2, 2001, pp. 5–39. The closed-form optimal-execution model this chapter's Section 1.2.2 contrasts with a learned, model-free alternative.
- Avellaneda, M., and S. Stoikov, "High-Frequency Trading in a Limit Order Book," *Quantitative Finance*, vol. 8, no. 3, 2008, pp. 217–224. The closed-form inventory-based market-making model Section 1.4.1 generalizes into a learned quoting policy.
- Buehler, H., L. Gonon, J. Teichmann, and B. Wood, "Deep Hedging," *Quantitative Finance*, vol. 19, no. 8, 2019, pp. 1271–1291. The paper that introduced Section 1.4.3's risk-measure-based, neural-network approach to hedging under transaction costs and other frictions.
- Abbeel, P., and A. Y. Ng, "Apprenticeship Learning via Inverse Reinforcement Learning," *Proceedings of the Twenty-First International Conference on Machine Learning*, 2004, pp. 1–8. The foundational paper on inverse reinforcement learning discussed in Section 1.3.4.

Wulin.Suo@Queensu.ca